

Microprocessor Interrupts

1. Purpose of Interrupts

Rules / Notes

Interfacing I/O Devices

- I/O devices (keyboards, printers) have very **low data transfer rates** compared to the microprocessor.
- To manage this speed mismatch, two primary methods are used:
 1. **Strobed or Polling Technique**
 2. **Interrupt Processing**

Concept

1. Strobed (Polling) Technique

The microprocessor **continuously checks** the status bit of an input device to see if new data has arrived.

- **Example:** Keyboard using the 82C55 PPI.
- Software polls the 82C55 and its **IBF (Input Buffer Full)** bit.
- If keyboard is typed at **1 cps**, the CPU wastes almost the entire second waiting between keystrokes.

Concept

2. Interrupt Processing

The microprocessor does **not** continuously check for input. It is allowed to execute other software in the meantime.

- **Mechanism:** When a user presses a key, the keyboard encoder **debounces the switch** and puts out one pulse.
- This signal tells the CPU that **“Data is Ready.”**
- CPU temporarily stops, jumps to the **Interrupt Service Routine (ISR)**, reads data, and then returns.

Concept

What is an Interrupt?

A signal that temporarily stops the normal execution of a program and forces the processor to run a special program called the **ISR**. After finishing, the processor returns to the main program.

2. Hardware Interrupt PINS

Rules / Notes

External Signal Pins

- **INTR (Interrupt Request):** Maskable interrupt generated by devices like keyboard, printer, etc. The CPU can ignore it if interrupts are disabled (**IF** = 0).
- **NMI (Non-Maskable Interrupt):** High-priority interrupt that **cannot be disabled**. Used for emergencies like power failure or memory parity error.
- **INTA (Interrupt Acknowledge):** Signal sent by the CPU to acknowledge that it has received an interrupt request from a device.

Concept

Control Flag Bits

- **IF (Interrupt Flag):** Interrupt enable flag.
 - **IF = 1:** CPU accepts maskable interrupts (INTR).
 - **IF = 0:** CPU ignores them.
- **TF (Trap Flag):** Single step flag used for debugging.
 - **TF = 1:** CPU executes one instruction and then interrupts (step-by-step execution).

Rules / Notes

Interrupt Initiation

An interrupt can be initiated by:

- **(i) Hardware:** External signals on pins.
- **(ii) Software:** Instructions like `INT n`.
- **(iii) Processor:** For internal errors (exceptions).

3. Interrupt Vector Table (IVT)

Concept

A structured memory list containing the starting addresses (vectors) of all ISRs.

Rules / Notes

IVT Characteristics

- **Total Size:** First 1024 bytes (256×4 bytes) = 1 KB.
- **Address Range:** 0000:0000H to 0000:03FFH.
- **Capacity:** 256 different 4-byte interrupt vectors.
- **General Allocation:** 32 Intel-Reserved (Type 0-31) | 224 User-defined.

Rules / Notes

IVT Memory Map (Detailed Segmentation)

- **Dedicated Interrupts (Types 0–4):**
 - **Memory Addresses:** 000H to 013H (Addresses 0–19).
 - Includes: Type 0 (Divide Error), Type 1 (Single Step), Type 2 (NMI), Type 3 (Breakpoint), Type 4 (Overflow).
- **Reserved Interrupts (Types 5–31):**
 - **Memory Addresses:** 014H to 07FH (Addresses 20–127).
 - Reserved by Intel for future/internal versions of their processors.
- **User Available (Types 32–255):**
 - **Memory Addresses:** 080H to 03FFH (Addresses 128–1023).
 - Available for hardware device designers and application software.

Concept

Interrupt Vector Format (Memory Layout)

Each vector is **4 bytes long** in real mode and contains the starting address of the interrupt service procedure.

- **First 2 bytes:** Offset address (IP).
- **Last 2 bytes:** Segment address (CS).

Storage Order (Little-endian):

CS : IP Byte	Physical Address	Memory Content
0000 : 0000H	00000H	IP Lower Byte (Offset)
0000 : 0001H	00001H	IP Higher Byte (Offset)
0000 : 0002H	00002H	CS Lower Byte (Segment)
0000 : 0003H	00003H	CS Higher Byte (Segment)

Example

Address Calculation for Type N

$$IP \text{ for ISR of Type N} = 4N$$

$$CS \text{ for ISR of Type N} = 4N + 2$$

Example for INT 5: Vector is at address 0014H ($4 \times 5 = 20 = 14H$).

4. Execution Flow & Instructions

Rules / Notes

Internal Hardware Sequence (The 7 Steps)

1. Push Flags onto stack.
2. Clear **IF** and **TF** flag (disable interference).
3. Push current **CS**.
4. Push current **IP** (return address).
5. Fetch new CS and IP from the IVT.
6. Execute the ISR.
7. **IRET (Interrupt Return)**: Used at the end of ISR. It restores the previous program's flags, CS, and IP from the stack so the CPU continues exactly where it left off.

Concept

Software Interrupt Instructions

Initiated by special instructions within the program itself.

- **INT n:** Executes interrupt number n . Vector specified by immediate byte.
 - *Example:* Vector address for **INT 5** is $4 \times 5 = 20$ (14H).
 - The vector is stored at memory addresses 0014H -- 0017H.
- **INTO:** Overflow trap. Executes ISR if **Overflow Flag (OF)** = 1. Used for arithmetic overflow checking.
- **INT 3:** Breakpoint trap. Helps debuggers pause programs at specific points.
 - **Opcode:** 1-byte instruction (0CCh).
- **BOUND:** Compares a register against 2 words of memory (DATA).
 - **Logic:** If $AX < DATA+1:DATA$ (Lower Limit) OR $AX > DATA+3:DATA+2$ (Upper Limit), **Type 5** interrupt occurs.
- **IRET (Interrupt Return):** Retrieves **4 bytes** (2 for IP, 2 for CS) return address and **2 bytes** Flag register from the stack.

5. Reserved Interrupt Types (Quick Reference)

Rules / Notes

Function of Reserved Interrupts

- **Type 0 to Type 4:** Dedicated interrupts.
- **Type 5 to Type 31:** Reserved interrupts.

Type	Name	Trigger Description
<i>Dedicated Interrupts (Type 0 – 4)</i>		
Type 0	Divide by Zero	Division result overflows destination; quotient too large.
Type 1	Single Step	Occurs after every instruction if $TF = 1$.
Type 2	NMI	Occurs when logic 1 is placed on the NMI input pin.
Type 3	Breakpoint	Execution of <code>INT 3 (0CCh)</code> for debugging.
Type 4	Overflow Trap	Generated by <code>INTO</code> instruction if $OF = 1$.
<i>Reserved/Internal Interrupts (Type 5 – 31)</i>		
Type 5	BOUND	Array index failed range check (<code>BOUND</code> instr).
Type 6	Invalid Opcode	CPU encounters an undefined or unknown machine code.
Type 8	Double Fault	Two separate interrupts occur during the same instruction.
Type 11	Segment Fault	Segment not present or invalid ($P=0$).

Quick Revision (One Look)

- Polling → Wasteful CPU waiting.
- Interrupt → Efficient reactive processing.
- IVT Size → 1 KB ($256 \text{ vectors} \times 4 \text{ bytes}$).
- Hardware Signals → `INTR` (Maskable), `NMI` (Non-Maskable).
- `IRET` → Pops **6 bytes** to restore state.
- Vector Formula → $4 \times N$ for IP, $4 \times N + 2$ for CS.

Exam Tips

Exam Rule:

- Hardware interrupts are initiated by pins (`INTR`, `NMI`).
- Software interrupts are initiated by instructions (`INT`, `INTO`, `BOUND`).
- Exceptions (Type 0, 6, 8, 11) are initiated by the processor itself.